

Informe Técnico General — Bitácora de Desarrollo con IA

Proyecto: Jill's Sandwich — Sistema de Gestión de Restaurante

Asignatura: Gestión de Desarrollo de Software — TP Integrador

Grupo: Dillinger Systems

Integrantes: Emiliano García · Giuliana Fragapane · Hiro Cruz

INDICE

1. Resumen del proyecto.....	4
Alcance de este informe.....	4
Repositorios y demo.....	4
Credenciales de demo.....	4
2. Arquitectura general.....	5
Módulos del sistema.....	5
Organización del backend (por features).....	5
Organización del frontend.....	6
3. Modelo de datos.....	6
Diagrama entidad-relación — visión general.....	6
Estructura de tablas.....	7
Relaciones entre entidades.....	7
Gestión de la creación de la base de datos desde el backend.....	8
Flujo de `setup_db.py` (instalación inicial).....	8
Flujo de `initialize_database()` (arranque del servidor).....	9
Configuración de conexión.....	10
Autenticación dual.....	11
4. Stack tecnológico.....	11
Frontend.....	11
Backend.....	11
Despliegue.....	11
5. Nuestro arsenal de herramientas de IA.....	12
6. Sinergia con la IA — Cómo nos ayudó a programar.....	12
6.1 Generación de código (Claude).....	12
6.2 Depuración y ajustes (Cursor).....	13
6.3 Documentación y entrega (Cursor).....	13
6.4 Prompts que funcionaron mejor en Cursor.....	14
7. Lecciones aprendidas y desafíos.....	14
7.1 Lo que funcionó bien.....	14
7.2 Donde la IA requirió intervención manual.....	14
7.3 Desafíos técnicos del proyecto.....	15
7.4 Reflexión sobre la eficiencia de la IA.....	15
8. Flujo de trabajo adoptado.....	16
Fase 1 — Definición del dominio.....	16
Fase 2 — Desarrollo y validación con IA.....	16
Fase 3 — Empaquetado y despliegue.....	17
Resumen del flujo.....	17
9. Pipelines CI/CD.....	17
9.1 Backend — Integración Continua (CI) y Despliegue (Railway).....	17

Arquitectura del pipeline CI.....	17
Despliegue continuo con Railway.....	18
Decisiones destacadas (CI).....	18
9.1.1 Herramientas del pipeline backend.....	19
9.2 Problemas encontrados y soluciones aplicadas.....	19
9.2.1 Imports aparentemente no utilizados.....	19
9.2.2 Errores de lint en el código existente.....	20
9.3 Frontend — Integración Continua y Despliegue Continuo (CI/CD).....	20
9.3.1 Pipeline de CI (`ci.yml`).....	21
9.3.2 Pipeline de CD (`deploy.yml`).....	22
9.3.3 Diferencias entre el build del CI y el build del deploy.....	23
9.3.4 Decisiones de diseño.....	24
9.3.5 Herramientas utilizadas.....	25
9.3.6 Flujo visual completo.....	25
9.4 Comparativa de pipelines.....	25
10. Checklist de entrega.....	26
11. Conclusión.....	26
Anexos.....	28
Anexo A — Diagrama ERD del sistema.....	28
Anexo B — Pipeline CI/CD del frontend (diagrama general).....	29
Anexo C — Flujo visual completo del pipeline frontend.....	30

1. Resumen del proyecto

Desarrollamos un sistema web full-stack para la administración de un restaurante ficticio llamado **Jill's Sandwich**. El objetivo del Trabajo Práctico Integrador fue demostrar cómo las herramientas modernas de IA pueden potenciar la productividad al codificar, documentar y desplegar una solución real en tiempo récord, sin recurrir a una arquitectura innecesariamente compleja.

El sistema está compuesto por tres capas:

Capa	Tecnología	Rol
Frontend	React + TypeScript + Vite	Panel de administración (SPA)
Backend	FastAPI + SQLAlchemy	API REST con autenticación JWT dual
Base de datos	MySQL	Persistencia relacional

Desde el panel web gestionamos pedidos, clientes, menú, mesas y ventas. Implementamos un modelo de seguridad con **dos tokens JWT**: uno de acceso general y otro específico para operaciones sensibles de ventas.

Alcance de este informe

Este documento unifica la bitácora de desarrollo de ambos repositorios (frontend y backend). No profundizamos en el proyecto base del cual se desprende el TP (automatización con n8n, WhatsApp, etc.), salvo donde impacta directamente en las entidades del dominio — por ejemplo, clientes identificados por teléfono o mesas con token QR.

Repositorios y demo

Recurso	Enlace
Frontend (GitHub)	https://github.com/EmiG4rcia/ProyectoDeGestionDeRestaurante-Front
Backend (GitHub)	https://github.com/EmiG4rcia/ProyectoDeGestionDeRestaurante-Back
Demo frontend	https://proyecto-de-gestion-de-restaurante.vercel.app
API desplegada (Railway / Swagger)	https://proyectodegestionderestaurante-back-producti.on.up.railway.app/docs

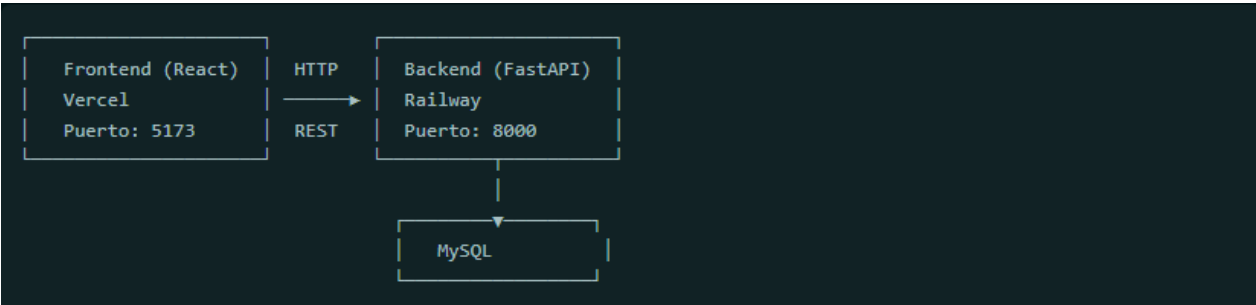
Credenciales de demo

Campo	Valor
-------	-------

Usuario	admin
Contraseña	admin1234
Contraseña de ventas	ventas1234

El **token de ventas** no es una credencial fija: se obtiene en runtime al ingresar la contraseña de ventas desde el panel (válido 15 minutos).

2. Arquitectura general



Módulos del sistema

Módulo	Frontend	Backend
Autenticación	Login, recuperación de contraseña	JWT (access token + sales token)
Dashboard	KPIs, estado de mesas, ítems no disponibles	Endpoints de resumen (/orders/summary)
Pedidos	CRUD, filtros, flujo de estados	/orders
Clientes	CRUD	/customers
Menú	CRUD, toggle de disponibilidad	/menu-items
Mesas	Estado disponible / ocupada	/tables
Ventas	Pagos, resumen de ingresos, doble verificación	/sales

Organización del backend (por features)

El backend es la capa de servicios del sistema: una API REST en **FastAPI** que persiste toda la información del restaurante en **MySQL** mediante **SQLAlchemy**. Expone endpoints para autenticación, gestión de pedidos, clientes, menú, mesas y ventas, y es consumida exclusivamente por el panel de administración. Además de los CRUDs de cada dominio, implementamos lógica de negocio como el flujo de estados de los pedidos, el cálculo de totales, la generación de QR para mesas y un modelo de seguridad con **dos tokens JWT** (acceso general y ventas).

Para mantener el código ordenado y extensible, adoptamos una estructura modular donde cada dominio tiene su propio paquete:

```
features/  
├─ auth/      → JWT, login, verificación de ventas  
├─ customers/ → CRUD de clientes  
├─ menu/      → CRUD del menú  
├─ orders/    → Pedidos y estados  
├─ sales/     → Pagos y resumen de ventas  
└─ tables/    → Mesas y generación de QR
```

Cada feature contiene `models.py`, `schemas.py`, `service.py` y `router.py`. La capa `core/` centraliza configuración (`.env`), conexión a base de datos, seguridad JWT y dependencias de autenticación.

Organización del frontend

El frontend es el **panel de administración** del restaurante: una SPA en **React + TypeScript** que permite al administrador gestionar el día a día del local desde el navegador. A través de módulos dedicados (dashboard, pedidos, clientes, menú, mesas y ventas), visualizamos KPIs en tiempo real, creamos y modificamos registros, y controlamos el flujo operativo sin tocar la base de datos directamente.

Toda la comunicación con el servidor pasa por la API REST: usamos **Axios** con interceptores JWT para enviar los tokens de autenticación, y **TanStack Query** para cachear y sincronizar los datos del servidor. Las operaciones sensibles de ventas requieren una **dobles verificación** con el `sales_token`, que el frontend solicita y gestiona de forma transparente para el usuario.

La estructura del código sigue una organización por responsabilidades:

```
src/  
├─ api/          # Clientes HTTP (axios) por dominio  
├─ auth/         # Contexto de autenticación y hooks  
├─ components/   # UI reusable (botones, modales, layout, tabla)  
├─ hooks/        # Custom hooks con React Query  
├─ pages/        # Páginas por ruta  
├─ routes/       # Configuración de React Router  
└─ types/        # Interfaces TypeScript compartidas
```

3. Modelo de datos

El diagrama entidad-relación (ERD) del sistema fue diseñado con asistencia de Claude y constituye la **fuentes de verdad conceptual** que guía la consistencia entre backend, frontend y base de datos. A partir de ese modelo definimos las entidades del dominio del restaurante, sus atributos, tipos de datos y las relaciones entre tablas antes de generar cualquier línea de código.

Ver [Anexo A — Diagrama ERD del sistema](#anexo-a--diagrama-erd-del-sistema).

Diagrama entidad-relación — visión general

El ERD modela el ciclo completo de operación de un restaurante con pedidos digitales: desde la identificación del cliente y la mesa, pasando por el menú y el carrito de compra, hasta el pedido

confirmado y su pago. Incluye además tablas orientadas al canal de mensajería del proyecto base (`sessions` y `message_logs`), que soportan flujos conversacionales asíncronos (por ejemplo, vía WhatsApp con n8n).

A continuación detallamos cada tabla del sistema y su rol dentro del dominio.

Estructura de tablas

Tabla	Atributos principales	Rol en el sistema
<code>`admin_users`</code>	id (PK), username (UK), password_hash, sales_password_hash, recovery_code_hash, created_at, last_login	Gestiona los usuarios administradores con credenciales de acceso y contraseña de ventas (ambas hasheadas con bcrypt)
<code>`customers`</code>	id (PK), phone_number (UK), name, created_at	Almacena clientes identificados por teléfono — clave natural del canal de mensajería
<code>`tables`</code>	id (PK), table_number (UK), qr_token (UK), status (enum)	Gestiona mesas físicas con estado (available / occupied) y token QR para pedidos en mesa
<code>`sessions`</code>	id (PK), phone_number (UK), customer_id (FK), table_id (FK), current_state, cart (json), last_activity	Rastrea interacciones activas: vincula cliente y mesa, mantiene una máquina de estados y un carrito temporal en JSON
<code>`orders`</code>	id (PK), customer_id (FK), table_id (FK), status (enum), total_amount, created_at, updated_at	Registra pedidos confirmados con su flujo de estados y monto total
<code>`menu_items`</code>	id (PK), category, name, description, price, is_available, popularity_score	Catálogo de platos y bebidas del restaurante
<code>`order_items`</code>	id (PK), order_id (FK), menu_item_id (FK), quantity, unit_price	Tabla intermedia: desglosa los ítems y cantidades de cada pedido, con precio unitario al momento de la orden
<code>`payments`</code>	id (PK), order_id (FK), amount, method (enum), status (enum), created_at	Registra transacciones financieras vinculadas a pedidos
<code>`message_logs`</code>	id (PK), phone_number, direction (enum), message_body, session_state, timestamp	Auditoría de mensajes entrantes/salientes del canal conversacional (tabla aislada, sin FKs)

Relaciones entre entidades

```
customers —(1:N)→ orders      "places"
customers —(1:N)→ sessions    "has"
tables —(1:N)→ orders         "hosts"
tables —(1:N)→ sessions       "linked to"
orders —(1:N)→ order_items    "contains"
menu_items —(1:N)→ order_items "included in"
orders —(1:N)→ payments       "paid via"
```

- Un **cliente** puede tener muchas **siones** activas y muchos **pedidos** históricos.

- Una **mesa** puede estar vinculada a varias **sesiones** y albergar múltiples **pedidos** a lo largo del tiempo.
- Un **pedido** contiene muchos **order_items**, cada uno referenciando un **menu_item** con cantidad y precio unitario congelado.
- Un **pedido** puede tener uno o más **pagos** asociados (pagos parciales o divididos).

Gestión de la creación de la base de datos desde el backend

No utilizamos un sistema de migraciones externo (como Alembic). En su lugar, gestionamos el esquema de la base de datos directamente desde el backend mediante **SQLAlchemy ORM**: cada entidad del dominio se define como una clase en `features/*/models.py`, y esas clases son la **única fuente de verdad** para la estructura de tablas en MySQL.

El mecanismo central es `Base.metadata.create_all(bind=engine)`, que lee todos los modelos registrados y crea las tablas que aún no existen. Para que SQLAlchemy los detecte, importamos explícitamente cada modelo antes de invocar `create_all` — tanto en `setup_db.py` como en `main.py`. Estos imports son necesarios por efecto secundario (registro en `Base.metadata`), aunque no se referencien directamente en el código.

Disponemos de **dos vías** para inicializar la base de datos, según el contexto:

Vía	Archivo	Cuándo la usamos
Setup completo (una sola vez)	<code>setup_db.py</code>	Primera instalación en un entorno nuevo
Inicialización automática	<code>main.py</code> → <code>lifespan</code> → <code>initialize_database()</code>	Cada vez que arranca el servidor FastAPI

Flujo de `setup\_db.py` (instalación inicial)

Ejecutamos `python setup_db.py` una sola vez al configurar un entorno nuevo. El script realiza cinco pasos secuenciales:

1. **Crear la base de datos** — Conecta a MySQL sin nombre de BD y ejecuta `CREATE DATABASE IF NOT EXISTS` con `charset utf8mb4`.
2. **Crear las tablas** — Importa todos los modelos y ejecuta `Base.metadata.create_all(bind=engine)`.
3. **Seed del menú** — Inserta 12 ítems de ejemplo (hamburguesas, sandwiches, pizzas, etc.) si la tabla está vacía.
4. **Seed de mesas** — Inserta 3 mesas con `qr_token` únicos si la tabla está vacía.
5. **Crear administrador** — Solicita interactivamente usuario, contraseña y contraseña de ventas; genera el código de recuperación.

Todos los pasos de seed verifican si ya existen registros antes de insertar, por lo que el script es **idempotente**: podemos ejecutarlo varias veces sin duplicar datos.

Flujo de `initialize_database()` (arranque del servidor)

Al iniciar la API con `fastapi dev main.py`, el hook `lifespan` de FastAPI invoca `initialize_database()` automáticamente. Esta función:

6. Ejecuta `Base.metadata.create_all` para garantizar que las tablas existan.
7. Si `menu_items` o `tables` están vacías, aplica el mismo seed de menú y mesas que `setup_db.py`.

De esta forma, en desarrollo no necesitamos correr el setup manual cada vez: el servidor se auto-configura al levantar. La creación del **primer administrador**, en cambio, requiere intervención explícita — ya sea mediante `setup_db.py` (modo interactivo) o el script auxiliar `scripts/seed_admin.py` (con argumentos por línea de comandos).



Configuración de conexión

La cadena de conexión se define en la variable de entorno `DATABASE_URL` (archivo `.env`), con el formato:

El engine de SQLAlchemy en `core/database.py` usa `pool_pre_ping` y `pool_recycle` para mantener conexiones estables con MySQL, tanto en desarrollo local como en despliegue con base de datos en la nube.

Autenticación dual

Token	Tipo JWT	Duración	Uso
access_token	type: access	480 min	Lectura, menú, mesas, cambio de estados
sales_token	type: sales	15 min	Crear/eliminar pedidos, clientes y pagos

Esta separación nos permitió implementar una **doble verificación** para operaciones sensibles sin complicar el login diario del administrador.

4. Stack tecnológico

Frontend

Tecnología	Uso
[React 19](https://react.dev/)	UI declarativa con componentes funcionales
[TypeScript](https://www.typescriptlang.org/)	Tipado estático
[Vite 8](https://vite.dev/)	Bundler y dev server
[Tailwind CSS 4](https://tailwindcss.com/)	Estilos utility-first
[React Router 7](https://reactrouter.com/)	Navegación SPA
[TanStack Query 5](https://tanstack.com/query)	Cache y sincronización de datos del servidor
[Axios](https://axios-http.com/)	Cliente HTTP con interceptores JWT

Backend

Tecnología	Uso
[FastAPI](https://fastapi.tiangolo.com/)	Framework web y documentación OpenAPI
[SQLAlchemy 2](https://www.sqlalchemy.org/)	ORM y mapeo de entidades
[MySQL](https://www.mysql.com/)	Base de datos relacional
[PyMySQL](https://pypi.org/project/PyMySQL/)	Driver de conexión
[Pydantic Settings](https://docs.pydantic.dev/latest/concepts/pydantic-settings/)	Configuración vía <code>.env</code>
[python-jose](https://python-jose.readthedocs.io/)	Tokens JWT
[passlib + bcrypt](https://passlib.readthedocs.io/)	Hash de contraseñas
[Uvicorn](https://www.uvicorn.org/)	Servidor ASGI

Despliegue

Servicio	Plataforma	Variable clave
Frontend	Vercel	VITE_API_URL → URL del backend en Railway

Backend	Railway	CORS_ORIGINS → URL del frontend en Vercel
---------	---------	---

5. Nuestro arsenal de herramientas de IA

Herramienta	Versión / Plan	Uso principal
Claude (chat web / Claude Code)	Pro	Generación del código frontend y backend, diseño del ERD, estructura del proyecto
Cursor	Agent mode	Documentación, depuración, CI/CD, scripts operativos y preparación para despliegue

Como grupo, definimos con claridad en qué contextos preferimos usar cada herramienta:

Herramienta	Contexto preferido	Por qué
Claude (chat web)	Empezar un proyecto desde cero	Es nuestro puntapié inicial. Destaca en diagramar estructura, clases y conexiones entre archivos a partir de un prompt conciso.
Cursor (agente en IDE)	Documentación, depuración y tareas mecánicas	Integrado en el editor: lee archivos, ejecuta terminal y modifica código sin salir del entorno de desarrollo.

En resumen: **Claude construye, Cursor mantiene, documenta y empaqueta.**

6. Sinergia con la IA — Cómo nos ayudó a programar

6.1 Generación de código (Claude)

El desarrollo del código base — tanto frontend como backend — fue impulsado principalmente por Claude, aprovechando el **contexto previo del dominio del restaurante** que ya estaba cargado en la conversación. Por eso, los prompts iniciales fueron **intencionalmente simples**:

Prompt inicial — frontend (reconstruido):

Necesitamos crear una interfaz web para el administrador del restaurante (panel de gestión de pedidos, clientes, menú, mesas y ventas).

Prompt inicial — backend (reconstruido):

Necesitamos la API REST en FastAPI para el panel de administración del restaurante. Debe cubrir autenticación, pedidos, clientes, menú, mesas y ventas, usando SQLAlchemy con MySQL y JWT para proteger los endpoints.

Tareas donde la IA fue más útil:

Área	Frontend	Backend
Estructura del proyecto	Vite + React + TypeScript	Arquitectura por features

UI y lógica	Componentes, modales, hooks con React Query	Modelos SQLAlchemy alineados con el ERD
Integración	Axios + interceptores JWT, rutas protegidas	Schemas Pydantic, servicios, routers REST
Seguridad	Contexto de auth con doble token	JWT dual (access + sales), bcrypt
Consistencia	Tipos TypeScript alineados con la API	Enums, filtros, totales calculados en servicio
Extras	Tailwind CSS, routing SPA	Generación de QR para mesas

Como grupo, pudimos comprobar de primera mano la increíble capacidad de esta IA para armar proyectos de pequeño y mediano alcance de una forma altamente eficiente y sin necesidad de ser excesivamente específico o explicativo a la hora de redactar un prompt. Podría decirse que la IA por sí misma tiene una concepción bastante acertada sobre las convenciones estándar que se tienen en el desarrollo de software y, en específico, en el desarrollo de aplicaciones Web.

Los prompts que **mejor funcionaron** con Claude fueron los orientados a la **generación de la estructura del proyecto**: Claude demostró ser muy eficiente a la hora de diagramar clases, archivos y cómo estos deberían estar conectados entre sí.

6.2 Depuración y ajustes (Cursor)

El código generado por Claude **funcionó desde las pruebas locales iniciales** sin errores críticos. El flujo consistió principalmente en probar la aplicación y pedir **ajustes iterativos** (UI, comportamiento, textos) cuando fue necesario.

Durante la **preparación para producción**, surgieron correcciones adicionales que resolvimos con **Cursor**:

- Errores de lint y variables no utilizadas (`OrderCreateModal` en el frontend)
- Refactor de `AuthContext` para compatibilidad con Fast Refresh
- 26 errores de lint en el backend (imports duplicados/no usados) corregidos con Ruff
- Iteración del workflow `deploy.yml` (instalación de pnpm, Node y Vercel CLI)
- Script `scripts/seed_admin.py` para crear el primer administrador (la función existía pero no estaba expuesta por API)

Estimación de tiempo:

Componente	Con IA	Manual (estimado)
Frontend	~1 h 30 min	~9 horas
Backend	< 2 horas	≥ 5 horas
Total aproximado	~3 h 30 min	~14 horas

6.3 Documentación y entrega (Cursor)

Cursor intervino en la fase de **empaquetado profesional** del TP:

- README completo de frontend y backend
- Informes técnicos (parciales y este documento unificado)
- requirements.txt, .env.example, .gitignore (backend)
- GitHub Actions: CI en backend, CI/CD en frontend
- Variables de entorno para despliegue (VITE_API_URL, secrets de Vercel)
- vercel.json para routing SPA
- Planificación de las etapas de armado y entrega según consignas del TP
- Despliegue del frontend en Vercel — completado
- Despliegue del backend en Railway — completado

6.4 Prompts que funcionaron mejor en Cursor

Los prompts orientados a **crear documentación** y a **ir modificándola según lo que necesitábamos** en cada iteración fueron los más productivos. Pedirle a Cursor que explore el proyecto, arme README e informe técnico, y luego refinar el contenido con correcciones puntuales (por ejemplo, ajustar el tono a trabajo grupal o eliminar referencias irrelevantes) resultó mucho más eficiente que redactar toda la documentación manualmente.

7. Lecciones aprendidas y desafíos

7.1 Lo que funcionó bien

- **Contexto previo en la conversación:** Tener el dominio del restaurante y el ERD ya cargados en Claude nos permitió prompts iniciales minimalistas pero efectivos.
- **Capacidad abarcativa de Claude:** Cumple órdenes específicas y, al mismo tiempo, **rellena eficientemente los huecos** que dejamos al promptear (tipos, hooks, manejo de errores, estructura de carpetas, enums, filtros).
- **Código funcional a la primera:** Tanto el frontend como el backend corrieron sin errores críticos desde la primera iteración, lo que nos ahorró tiempo de depuración.
- **Consistencia con el ERD:** Al compartir el modelo de datos, la IA mantuvo coherencia entre frontend, backend y base de datos.
- **Arquitectura por features (backend):** La separación en módulos independientes facilitó entender, extender y documentar cada dominio.
- **Velocidad medible:** Completamos un proyecto full-stack funcional en aproximadamente **3 horas y media** con IA, frente a una estimación de **~14 horas** de forma manual.

7.2 Donde la IA requirió intervención manual

- **Ajustes iterativos de UI/UX:** No fueron correcciones de errores, sino refinamientos pedidos explícitamente (cambios visuales, flujos, textos).

- **Credenciales iniciales (backend):** La función `create_first_admin` existía en el código pero no estaba expuesta por API ni documentada; lo resolvimos con un script de seed.
- **Preparación para producción:** Faltaban `requirements.txt`, `.gitignore`, pipelines CI, guías de despliegue y URLs hardcodeadas a `localhost` — resueltos progresivamente con Cursor.
- **Base de datos:** La conexión MySQL, creación de tablas y variables de entorno requieren configuración manual fuera del alcance del chat.
- **Despliegue:** Configuración de variables de entorno, secrets de GitHub/Vercel, ajustes de CORS y iteraciones del pipeline de deploy requirieron varias vueltas antes de lograr un deploy estable del frontend.

7.3 Desafíos técnicos del proyecto

- **Autenticación dual:** Diseñar dos tokens JWT con distintos scopes y tiempos de expiración, y reflejarlo en frontend y backend.
- **Operaciones con distinto nivel de acceso:** Algunos endpoints usan `get_current_admin` y otros `get_sales_access` dentro del mismo router.
- **Integridad referencial:** Pedidos vinculados a clientes, mesas e ítems del menú con totales calculados en servicio.
- **CORS en despliegue:** Actualizar `allow_origins` en `main.py` con la URL real del frontend en Vercel.
- **Integración cross-origin:** Mantener sincronizados dos repositorios separados con documentación y contratos de API compartidos.
- **Despliegue con MySQL externo:** Configurar `DATABASE_URL` en la nube y mantener la base sincronizada con los modelos.

7.4 Reflexión sobre la eficiencia de la IA

Para un proyecto de esta escala (CRUD + auth + lógica de negocio moderada + panel SPA), Claude demostró ser **altamente eficiente** cuando:

8. El ERD y los flujos de negocio estaban claros antes de generar código
9. Confiamos en la IA para completar detalles no explicitados (schemas, enums, filtros, hooks)
10. Reservamos Cursor para depuración, documentación y empaquetado profesional
11. Iteramos con pedidos de cambio puntuales en lugar de reescribir desde cero

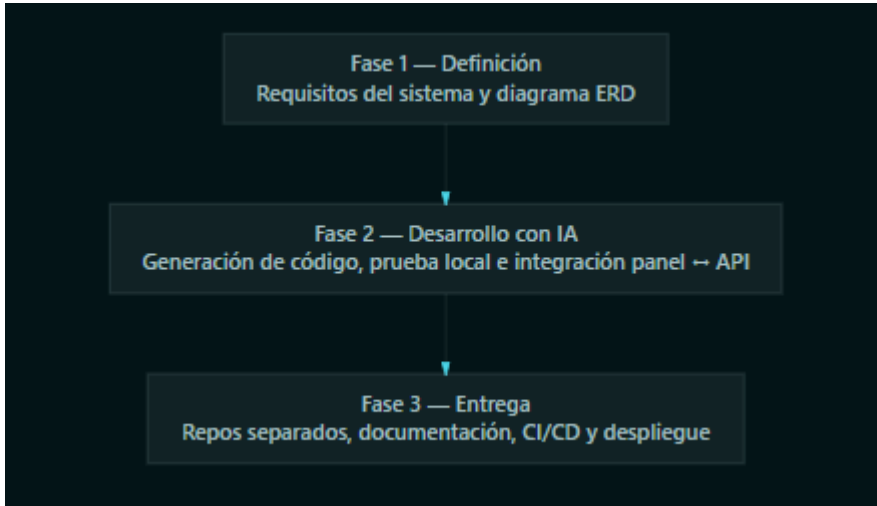
La clave fue **no sobre-explicar el prompt** cuando el contexto ya estaba cargado, y confiar en la capacidad de Claude para diagramar la estructura del proyecto de forma coherente.

La IA no reemplaza el criterio del equipo, pero nos permitió enfocarnos en decisiones de arquitectura (como la autenticación dual) mientras delegábamos la implementación repetitiva de CRUDs, schemas, routers y componentes.

8. Flujo de trabajo adoptado

A lo largo del TP seguimos un proceso en **tres fases** bien diferenciadas: primero definimos el dominio, luego generamos y validamos el código con IA, y por último empaquetamos y desplegamos el proyecto con herramientas integradas al IDE. El diagrama resume ese recorrido; debajo detallamos cada etapa.

Diagrama (Mermaid — representación textual)



Fase 1 — Definición del dominio

Antes de escribir código, acordamos **qué debía resolver el sistema** y diseñamos el **modelo de datos** (ERD). Este paso fue clave: al tener el dominio del restaurante y las entidades ya definidas, pudimos usar prompts breves en Claude sin tener que explicar el contexto desde cero en cada iteración.

Fase 2 — Desarrollo y validación con IA

Con el ERD como guía, pedimos a **Claude** la generación del frontend (React) y del backend (FastAPI). Luego **probamos en local**: levantamos la API, conectamos MySQL y verificamos que el panel consumiera los endpoints correctamente.

Si algo no funcionaba o queríamos refinarlo, entrábamos en un **ciclo de ajustes**:

Tipo de tarea	Herramienta que usamos	Ejemplos
Cambios de lógica, estructura o nuevas features	Claude	Modificar flujos de pedidos, agregar endpoints
Depuración, lint, refactors menores	Cursor	Corregir imports, AuthContext, errores de build

Cada corrección volvía a pasar por **prueba local** hasta que el sistema quedaba estable.

Fase 3 — Empaquetado y despliegue

Una vez validado el código, separamos el proyecto en **dos repositorios** (frontend y backend) y usamos **Cursor** para la fase de entrega profesional:

12. **Documentación** — README, informes técnicos y guías de instalación.

13. **Automatización** — pipelines de GitHub Actions (CI en backend, CI/CD en frontend).

14. **Scripts operativos** — seed de administrador, variables de entorno y configuración de CORS.

15. **Despliegue** — frontend en Vercel y backend en Railway (completados).

Resumen del flujo

Fase	Objetivo	Herramienta principal	Resultado
1 — Definición	Acordar qué construir	Claude (diseño del ERD)	Modelo de datos y requisitos claros
2 — Desarrollo	Generar y validar código	Claude + Cursor	Sistema funcional en local
3 — Entrega	Documentar, automatizar y publicar	Cursor	Repos listos para producción

9. Pipelines CI/CD

Configuramos pipelines con **GitHub Actions** en ambos repositorios. A continuación documentamos la arquitectura de cada uno y las decisiones de diseño que tomamos como grupo.

9.1 Backend — Integración Continua (CI) y Despliegue (Railway)

El pipeline de validación se define en `.github/workflows/ci.yml`. El **despliegue en producción** lo gestiona **[Railway](https://railway.com/)** de forma automática al detectar un push a `main`, sin un job de deploy explícito en GitHub Actions.

Componente	Responsable	Mecanismo
CI (validación)	GitHub Actions	Jobs: lint, import-check, security
CD (deploy)	Railway	Deploy automático vía webhook de GitHub
Ejecución	Uvicorn + FastAPI	Definida en Procfile

Arquitectura del pipeline CI

Consta de **tres jobs independientes** que se ejecutan en **paralelo**:

Diagrama (Mermaid — representación textual)



Job	Herramienta	Qué valida
Lint & Format	[Ruff](https://docs.astral.sh/ruff/)	Estilo de código y errores estáticos (Python 3.11 y 3.12)
Import & App Verification	Smoke test	Que la app FastAPI inicialice sin errores (Python 3.11 y 3.12)
Dependency Security Audit	[pip-audit](https://pypi.org/project/pip-audit/)	Vulnerabilidades conocidas en dependencias (Python 3.12)

En **pull requests**, solo corren estos tres jobs — no hay deploy asociado. En **push a `main`**, Railway despliega en paralelo mientras GitHub Actions valida el código.

Despliegue continuo con Railway

Railway está conectado al repositorio del backend. Cada push a `main` dispara automáticamente:

```

Push a main → Railway detecta el cambio → clona el repo
→ pip install -r requirements.txt → ejecuta Procfile
→ uvicorn main:app --host 0.0.0.0 --port $PORT
→ dominio público con HTTPS
  
```

El Procfile indica cómo iniciar la aplicación:

```
web: uvicorn main:app --host 0.0.0.0 --port ${PORT:-8000}
```

FastAPI define las rutas y la lógica de negocio; **Uvicorn** es el servidor ASGI que recibe las peticiones HTTP y se las delega a FastAPI. Railway abstrae el reverse proxy, TLS y la gestión de procesos — nosotros solo necesitamos el Procfile y las variables de entorno (`DATABASE_URL`, `SECRET_KEY`, `CORS_ORIGINS`, etc.).

API en producción: <https://proyectodegestionderestaurante-back-production.up.railway.app/docs> (Swagger) · URL base: <https://proyectodegestionderestaurante-back-production.up.railway.app>

Decisiones destacadas (CI)

- **Paralelismo entre jobs:**

Los tres jobs no tienen dependencias entre sí y se ejecutan en paralelo. Esto reduce el tiempo total del pipeline: en lugar de esperar que cada job termine secuencialmente, los tres corren simultáneamente y el pipeline termina cuando el más lento finaliza.

- **Matrix de versiones de Python:**

Los jobs de lint e import-check utilizan una matrix con Python 3.11 y 3.12. Esto garantiza que el código sea compatible con ambas versiones, lo cual es relevante cuando se trabaja en equipos donde los desarrolladores pueden tener distintas versiones instaladas localmente.

El job de security audit no necesita matrix porque audita las dependencias (paquetes de terceros), no el código propio.

- **Smoke test con SQLite:**

En lugar de levantar un contenedor de MySQL en CI (lo cual agregaría complejidad y tiempo), el smoke test usa SQLite como base de datos temporal. Esto funciona porque el objetivo es verificar que la app inicie, no que ejecute queries de producción. SQLAlchemy abstrae el motor de base de datos, haciendo posible esta sustitución sin modificar el código de la aplicación.

- **Separación de responsabilidades:**

Cada job tiene un propósito claro y distinto:

Job	Pregunta que responde
Lint & Format	¿El código cumple con los estándares de estilo?
Import & App Verification	¿La aplicación puede inicializarse sin errores?
Dependency Security Audit	¿Las dependencias tienen vulnerabilidades conocidas?

9.1.1 Herramientas del pipeline backend

Herramienta	Versión	Rol en el pipeline
GitHub Actions	v2 (hosted runners)	Plataforma de ejecución del CI
actions/checkout	v4	Clonado del repositorio
actions/setup-python	v5	Instalación de Python en el runner
Ruff	Última disponible	Linter y verificador de formato
pip-audit	Última disponible	Auditoría de seguridad de dependencias
SQLite	Incluido en Python	Base de datos temporal para smoke test
Railway	PaaS cloud	Deploy automático y hosting del backend
Uvicorn	0.47.x	Servidor ASGI en producción

Nota: A diferencia del frontend (dos workflows en GitHub Actions), el backend concentra la **validación en Actions** y el **deploy en Railway**. Railway despliega al detectar el push; no espera a que el CI termine — el código que llega a main ya pasó CI en la etapa de PR.

9.2 Problemas encontrados y soluciones aplicadas

9.2.1 Imports aparentemente no utilizados

Problema: Ruff reportaba 26 errores, incluyendo imports marcados como no utilizados en main.py y setup_db.py .

Causa raíz: SQLAlchemy requiere que los modelos sean importados para que se registren en Base.metadata . Estos imports son necesarios como efecto secundario, aunque Ruff no puede detectar ese patrón.

Solución: Configuramos `ruff.toml` con `per-file-ignores` para suprimir la regla F401 únicamente en esos dos archivos, manteniendo la regla activa en el resto del proyecto.

9.2.2 Errores de lint en el código existente

Problema: El código generado inicialmente contenía imports duplicados e imports no utilizados en varios routers y servicios.

Solución:

-Se corrigieron los 26 errores de lint identificados por Ruff.

-Se reformatearon 31 archivos `.py` con `ruff format` .

- Se realizaron 2 commits separados: uno para la configuración del CI y otro para las correcciones de estilo, manteniendo el historial limpio.

9.3 Frontend — Integración Continua y Despliegue Continuo (CI/CD)

Configuramos un pipeline de **Integración Continua (CI)** y **Despliegue Continuo (CD)** con GitHub Actions para validar automáticamente cada cambio y desplegar a Vercel en producción. El frontend utiliza **dos workflows independientes** en `.github/workflows/`:

Archivo	Propósito	Trigger
ci.yml	Validación: lint + build	Push y PR a main / master
deploy.yml	Despliegue a Vercel	Solo push a main / master

El diagrama siguiente muestra cómo se disparan los workflows y qué pasos ejecuta cada uno. En un **push a `main`**, CI y Deploy corren **en paralelo**; en un **pull request**, solo corre el CI.

Ver [Anexo B — Pipeline CI/CD del frontend (diagrama general)](#anexo-b--pipeline-cicd-del-frontend-diagrama-general).

Resumen por workflow:

	CI (`ci.yml`)	Deploy (`deploy.yml`)
Cuándo corre	Push y PR a main / master	Solo push a main / master
Objetivo	Verificar que el código compila y pasa lint	Publicar la app en Vercel
Pasos	checkout → pnpm + Node → install → lint → build	checkout → pnpm + Node → Vercel CLI → pull → build → deploy
Resultado	CI exitoso o fallido	App en producción

9.3.1 Pipeline de CI (.ci.yml)

Propósito

Validar que el código TypeScript/React compila correctamente y cumple con los estándares de calidad definidos por ESLint. Este pipeline actúa como **guardián del código** ante cada cambio.

Eventos de disparo

```
on:
  push:
    branches: [main, master]
  pull_request:
    branches: [main, master]
```

Se ejecuta tanto en push directo como en pull requests hacia main/master, lo que nos permite detectar problemas **antes** de integrar el código.

Estructura del job

El CI consta de un **único job secuencial** con 6 pasos:

Paso	Comando	Propósito
1	actions/checkout@v4	Clonar el repositorio
2	pnpm/action-setup@v4	Instalar pnpm v9
3	actions/setup-node@v4	Instalar Node.js 20 con caché de pnpm
4	pnpm install --frozen-lockfile	Instalar dependencias de forma reproducible
5	pnpm lint	Ejecutar ESLint
6	pnpm build	Compilar la aplicación con Vite

Los pasos son secuenciales por diseño: si el lint falla, no se intenta el build. Esto ahorra tiempo de ejecución y da feedback más claro.

Instalación con `--frozen-lockfile`

Usamos `--frozen-lockfile` para garantizar reproducibilidad en el pipeline:

- **Sin el flag:** si `package.json` tiene `"react": "^19.0.0"` y sale una versión nueva, pnpm podría instalar una distinta a la del entorno de desarrollo.
- **Con el flag:** pnpm rechaza la instalación si `pnpm-lock.yaml` no coincide exactamente con `package.json`, forzando a actualizar el lockfile explícitamente.

Así el CI siempre valida **exactamente las mismas versiones** que usamos en desarrollo.

Configuración de ESLint

El linter está definido en `eslint.config.js` con un perfil estricto:

```
extends: [
  js.configs.recommended,           // Reglas base de JavaScript
  tseslint.configs.recommended,     // Reglas de TypeScript
  reactHooks.configs.flat.recommended, // Reglas de React Hooks
  reactRefresh.configs.vite,        // Compatibilidad con Vite HMR
]
```

Este conjunto de reglas detecta, entre otros:

- Variables e imports no utilizados
- Violaciones de React Hooks (llamadas condicionadas, orden incorrecto)
- Componentes incompatibles con Vite Fast Refresh

Durante la preparación para producción corregimos varios de estos errores (por ejemplo, en `OrderCreateModal` y `AuthContext`).

Build con variable de entorno

```
env:
  VITE_API_URL: https://example-api.onrender.com
```

El build en CI necesita una URL de API porque Vite inyecta las variables `VITE_*` en tiempo de compilación. En el entorno de CI usamos una **URL placeholder**; el objetivo no es generar un build desplegable, sino verificar que la compilación no tiene errores (TypeScript, imports, JSX).

9.3.2 Pipeline de CD (`deploy.yml`)

Propósito

Desplegar automáticamente la aplicación a **Vercel** en producción cada vez que se hace push a las ramas principales.

Eventos de disparo

```
on:
  push:
    branches: [main, master]
```

A diferencia del CI, el deploy **no se ejecuta en pull requests** — solo cuando el código ya fue integrado a `main/master`.

Estructura del job

El deploy consta de **7 pasos**:

Paso	Comando	Propósito
1	<code>actions/checkout@v4</code>	Clonar el repositorio
2	<code>pnpm/action-setup@v4</code>	Instalar pnpm v9
3	<code>actions/setup-node@v4</code>	Instalar Node.js 20

4	<code>npm i -g vercel@latest</code>	Instalar Vercel CLI
5	<code>vercel pull</code>	Descargar configuración del proyecto desde Vercel
6	<code>vercel build --prod</code>	Compilar con la configuración de producción
7	<code>vercel deploy --prebuilt --prod</code>	Subir el build precompilado a producción

Las tres fases del deploy

Fase 1 — Pull:

```
vercel pull --yes --environment=production --token=${{ secrets.VERCEL_TOKEN }}
```

Descarga la configuración del proyecto en Vercel: variables de entorno, preset del framework y ajustes de build. El flag `--yes` evita prompts interactivos, necesario en CI.

Fase 2 — Build:

```
vercel build --prod --token=${{ secrets.VERCEL_TOKEN }}
```

Compila la aplicación con la configuración descargada en la fase 1. A diferencia del build del CI (que usa una URL placeholder), este build usa las **variables reales** configuradas en el dashboard de Vercel — incluyendo `VITE_API_URL` apuntando al backend de producción.

Fase 3 — Deploy:

```
vercel deploy --prebuilt --prod --token=${{ secrets.VERCEL_TOKEN }}
```

Sube el build precompilado (`--prebuilt`) al entorno de producción (`--prod`). No recompila: usa los artefactos generados en la fase 2.

Secrets requeridos

Secret	Propósito
VERCEL_TOKEN	Token de autenticación para la API de Vercel
VERCEL_ORG_ID	Identificador de la organización/equipo en Vercel
VERCEL_PROJECT_ID	Identificador del proyecto en Vercel

Estos valores se configuran en GitHub → Settings → Secrets and variables → Actions y nunca se exponen en el código ni en los logs del pipeline.

9.3.3 Diferencias entre el build del CI y el build del deploy

Aspecto	CI (ci.yml)	Deploy (deploy.yml)
Herramienta de build	<code>pnpm build</code> (Vite directo)	<code>vercel build --prod</code> (CLI de Vercel)
API URL	Placeholder (example-api.onrender.com)	<code>https://proyectedegestionderestaurante-back-production.up.railway.app</code> (configurada en Vercel)

Propósito	Verificar que compila sin errores	Generar artefactos desplegables
Output	Se descarta	Se sube a Vercel
Variables de entorno	Definidas en el workflow	Descargadas de Vercel con vercel pull

Ambos procesos compilan la aplicación, pero con objetivos distintos: el CI pregunta "*¿compila?*"; el deploy pregunta "*¿está listo para producción?*".

9.3.4 Decisiones de diseño

Dos workflows separados en lugar de uno

Separamos CI y CD en dos archivos por tres razones:

16. **Triggers distintos:** el CI corre en push y PR; el deploy solo en push a `main`.
17. **Responsabilidades claras:** si falla el deploy, revisamos `deploy.yml`; si falla el lint, revisamos `ci.yml`.
18. **Independencia:** cada workflow puede evolucionar sin afectar al otro.

CI y deploy corren en paralelo

Cuando hay push a `main`, ambos workflows se disparan al mismo tiempo. El deploy **no depende** de que el CI pase primero.

Implicación: si el CI falla, el deploy podría igualmente subir código con errores a producción.

Justificación: el código que llega a `main` ya pasó CI durante la etapa de PR. El CI en push es una validación adicional, no la barrera principal. Un modelo más estricto (con `needs: build`) es posible, pero priorizamos simplicidad.

Job único secuencial (CI) vs. jobs paralelos (backend)

A diferencia del backend —que ejecuta tres jobs en paralelo—, el frontend usa un **único job secuencial**:

19. **Menos validaciones:** el front solo tiene lint y build; el back tiene lint, smoke test y auditoría de seguridad.
20. **Dependencia natural:** si el lint falla, el build también fallaría. Ejecutarlos en secuencia evita gastar minutos de CI en un build condenado a fallar.
21. **Setup compartido:** `pnpm` y `Node.js` se instalan una sola vez. Jobs paralelos requerirían setups separados.

Caché de dependencias


```

- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: 20
    cache: pnpm

```

La opción `cache: pnpm` almacena el directorio de dependencias entre ejecuciones. En la primera corrida, pnpm descarga todos los paquetes; en las siguientes, si `pnpm-lock.yaml` no cambió, se restauran desde caché — reduciendo significativamente el tiempo de instalación.

9.3.5 Herramientas utilizadas

Herramienta	Versión	Rol en los pipelines
GitHub Actions	v2 (hosted runners)	Plataforma de ejecución
actions/checkout	v4	Clonado del repositorio
pnpm/action-setup	v4	Instalación de pnpm
actions/setup-node	v4	Instalación de Node.js con caché
pnpm	v9	Package manager
Node.js	20 LTS	Runtime de JavaScript
ESLint	10.3.0	Linter de TypeScript/React
Vite	8.0.12	Bundler y compilador
Vercel CLI	Última disponible	Despliegue a producción

9.3.6 Flujo visual completo

El diagrama siguiente incluye las **rutas de fallo** del workflow de CI (lint o build fallido) y el flujo completo del deploy en paralelo.

Ver [Anexo C — Flujo visual completo del pipeline frontend](#anexo-c--flujo-visual-completo-del-pipeline-frontend).

En un **push a `main`**, ambos workflows arrancan al mismo tiempo: el de la izquierda publica en Vercel; el de la derecha valida lint y build. En un **pull request**, solo se ejecuta el flujo de CI (validación).

9.4 Comparativa de pipelines

Aspecto	Backend	Frontend
Jobs CI	3 en paralelo	1 secuencial (lint + build)
CD	Railway (automático al push)	Vercel vía <code>deploy.yml</code>
CI en PR	✅	✅
Plataforma de deploy	Railway	Vercel
Validaciones	Lint, smoke test, auditoría de seguridad	Lint, build
Matrix de versiones	Python 3.11 + 3.12	Node.js 20

10. Checklist de entrega

#	Requisito	Estado
1	Repo público en GitHub (frontend)	✓ https://github.com/EmiG4rcia/ProyectoDeGestionDeRestaurante-Front
2	Repo público en GitHub (backend)	✓ https://github.com/EmiG4rcia/ProyectoDeGestionDeRestaurante-Backend
3	Pipeline CI (backend)	✓ Configurado y documentado
4	Pipeline CI/CD (frontend)	✓ Configurado, desplegando y documentado
5	Pipeline CD (backend — Railway)	✓ Deploy automático en Railway
6	Informe de herramientas de IA	✓ Redactado (este documento)
7	Demo funcional online (frontend)	✓ https://proyecto-de-gestion-de-restaurante.vercel.app
8	API desplegada y accesible	✓ https://proyectodegestionderestaurante-back-production.up.railway.app/docs
9	Frontend conectado al backend en producción	✓ (VITE_API_URL + CORS ORIGINS)
10	README de calidad (front + back)	✓ Creados
11	Swagger funcional en /docs	✓ https://proyectodegestionderestaurante-back-production.up.railway.app/docs
12	Script de seed de admin	✓ Creado
13	Documentación general unificada	✓ Este informe

11. Conclusión

El TP Integrador de **Jill's Sandwich** demostró que el desarrollo asistido por IA no es solo una herramienta de autocompletado, sino un **multiplicador de productividad real**. Pasar de una estimación de ~14 horas a ~3 horas y media en un proyecto full-stack funcional es una diferencia concreta y medible para nuestro equipo.

La clave del éxito fue **definir el dominio y el modelo de datos primero**, y luego apoyarnos en la IA generativa para producir código coherente entre frontend, backend y base de datos. Claude construyó la base del proyecto con prompts concisos gracias al contexto previo cargado; Cursor complementó el proceso en la fase de **empaquetado profesional**: documentación, dependencias, scripts operativos, pipelines de CI/CD y despliegue.

Como grupo, adoptaríamos el mismo flujo en proyectos futuros: definir requisitos y ERD, usar Claude para la generación rápida del código, y reservar herramientas como Cursor para documentación, depuración y entrega. La IA no reemplaza nuestro criterio técnico, pero elimina

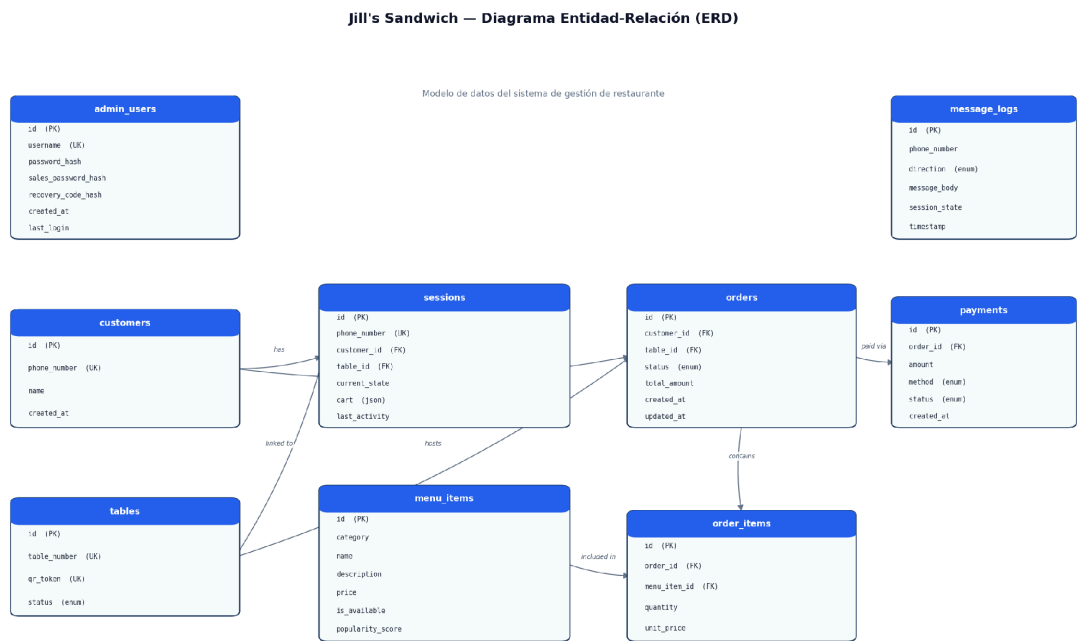
la fricción de las tareas repetitivas y nos permite enfocarnos en decisiones de arquitectura — como la autenticación dual — y en la experiencia de usuario.

El sistema quedó **desplegado y accesible en producción**: el panel en <https://proyecto-de-gestion-de-restaurante.vercel.app> y la API en <https://proyectodegestionderestaurante-back-production.up.railway.app/docs>, con pipelines de CI/CD documentados en la sección 9 de este informe.

Anexos

Las figuras referenciadas en el cuerpo del informe se presentan a continuación. Las figuras se incluyen embebidas en este documento; debajo de cada anexo incluimos el **pie de foto** sugerido.

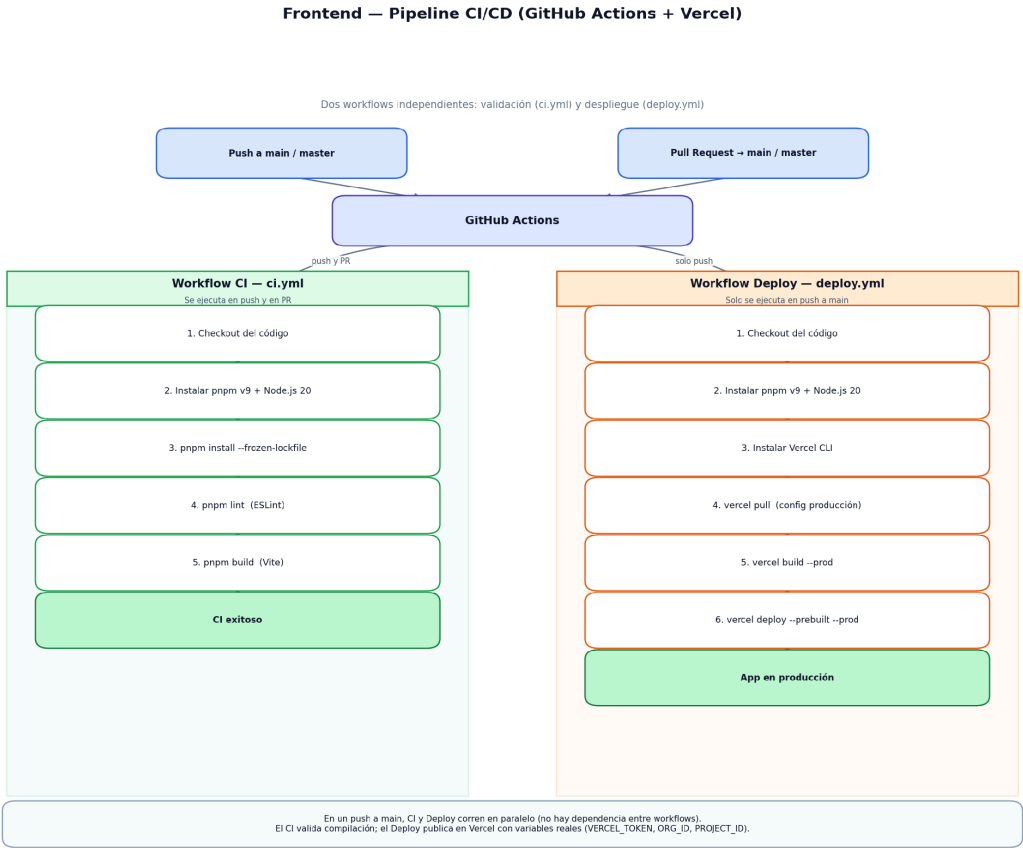
Anexo A — Diagrama ERD del sistema



Referenciado en Sección: 3 — Modelo de datos

Pie de foto: *Figura generada con Cursor. Diagrama entidad-relación (ERD) del sistema Jill's Sandwich: muestra las tablas del dominio (clientes, mesas, pedidos, ítems de menú, pagos, sesiones y logs de mensajes), sus atributos principales y las relaciones entre entidades.*

Anexo B — Pipeline CI/CD del frontend (diagrama general)

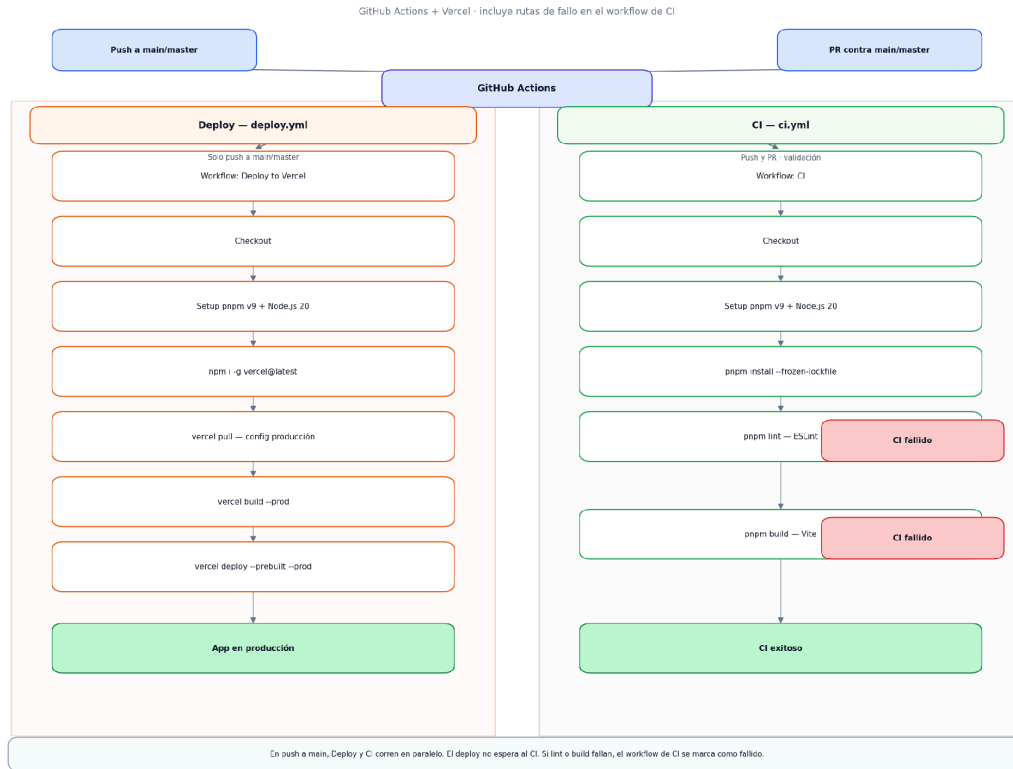


Referenciado en Sección: 9.3 — Frontend CI/CD

Pie de foto: *Figura generada con Cursor.* Esquema general del pipeline de integración y despliegue continuo del frontend: disparadores (push y pull request), workflows `ci.yml` y `deploy.yml`, y los pasos principales de validación (lint, build) y publicación en Vercel.

Anexo C — Flujo visual completo del pipeline frontend

Frontend — Flujo visual completo del pipeline CI/CD



Referenciado en Sección: 9.3.6 — Flujo visual completo

Pie de foto: *Figura generada con Cursor.* Diagrama detallado del pipeline frontend: ejecución en paralelo de CI y deploy ante un push a `main`, pasos secuenciales de cada workflow y rutas de fallo cuando lint o build no superan la validación.